

Neighborhoods, Ripley's K, etc.

Gabriel Arellano

2026-05-23

Introduction

Spatial neighborhood analyses are used to quantify how organisms, stems, species, functional groups, or other ecological units are arranged around focal individuals or locations. In forest ecology, these analyses are commonly used to evaluate local crowding, conspecific and heterospecific aggregation, spatial segregation, density dependence, habitat association, biomass accumulation around focal trees, and departures from spatial null expectations. Classical summaries include neighbor counts or densities within circles or rings, Ripley's K, and transformations such as Besag's L.

This tutorial introduces updated code for calculating neighborhood summaries at multiple spatial scales. The workflow separates several operations that were previously bundled together: identifying neighbors within a distance threshold, defining focal and reference classes, handling circles and annuli, estimating available neighborhood area near valid boundaries (of arbitrary shape, but typically the plot limits), summarizing abundance as counts of points or other metrics, and comparing observations with null expectations. This separation is intentional. It makes the calculations easier to inspect, modify, test, and extend.

Relative to the previous CTFS functions, the updated code is designed to be more modular and less tied to specific assumptions about rectangular plots, point classes, or abundance metrics. Focal points, neighbor points, filters, spatial boundaries, abundance metrics, and null models are supplied independently and explicitly by the user. Edge correction is handled for boundaries of arbitrary shape, using numerical area estimates. The same neighbor-identification step can support standard count-based analyses, biomass or basal-area summaries, ring-based calculations, Ripley-type summaries, and simulation-based null models. The practical trade-off is that the workflow gives the user more flexibility, but also more responsibility to define biologically meaningful focal classes, reference sets, spatial windows, metrics, and null expectations. The approach taken is very fast but demands a lot of memory, so this can be a practical limitation for large datasets too.

This is updated code to calculate abundances of neighbors at different scales, in circles and rings. It also contains instructions to compare observations with expectations, including Ripley's K and Besag's L.

List of old and new functions

The code here substitutes these original CTFS functions:

- “CalcRingArea” in `spatial > RipUvK.r`
- “partialcirclearea” in `spatial > RipUvK.r`
- “circlearea” in `spatial > RipUvK.r`
- “Annuli” in `spatial > RipUvK.r`
- “NDcount” in `spatial > NeighborDensityFun.r`
- “NeighborDensities” in `spatial > NeighborDensityFun.r`
- “RipUvK” in `spatial > RipUvK.r`

The new functions are:

- `fast_distances_up_to_r`: the workhorse to calculate distances from one set of focal points (query) to their neighbors (taken from a reference dataset), within a given threshold. This takes advantage of the very fast `dbscan::frNN` function, and will be called often from within other functions here.
- `boundary_around_xy`: an auxiliary function to estimate a (complex) boundary around a given set of points. This may or may not help much, but it is needed as a default in case the user does not provide a proper boundary.
- `core_vs_edge`: an auxiliary function that classifies a set of points into “out” (outside the provided boundary of arbitrary shape), “edge” (inside the boundary, but closer than a given distance to it), and “core” (inside the boundary and far from the edge). The main goal of this auxiliary function is to save computation by avoiding a numerical calculation for the area of full circles around “core” points. Also, it uses the “core” points to estimate the error associated with numerical area estimates derived from a grid.
- `make_grid_inside_boundary`: creates a regular grid within the provided boundary. The grid will be then used to count number of points within shapes, so it is a very general way of estimating areas of truncated circles near the (possibly complex) boundary.
- `guess_sufficient_grid_resolution`: the grid approach is flexible but, as it is a numerical estimate, it also carries some error. The `guess_sufficient_grid_resolution` estimates the resolution of the grid required to keep error constrained below a given tolerance (1% by default).
- ...

Filters, abundance metrics, null expectations, and classes to compare, are all outside these functions.

Code

The main function here is a function to calculate, fast, distances up to certain threshold. This is useful for the distances themselves and to identify neighbors.

```
# Fast calculation of distances up to a given radius:
fast_distances_up_to_r <- function(query, reference = NULL, radius, approx = 0) {

  # Parameters:
  # query: the set of focal points, around which we are going to look
  # reference: the set of potential neighbors; will default to be the same as "query"
  # radius: the size of the local neighborhood around the focal point
  # approx: precision parameter to be passed to dbscan::frNN

  # Reshape input
  if(is.null(reference)) reference <- query
  query <- as.matrix(query)
  reference <- as.matrix(reference)

  storage.mode(query) <- "double"
  storage.mode(reference) <- "double"

  # Calculate distances
  nn <- dbscan::frNN(
    x = reference,
    query = query,
    eps = radius,
    sort = FALSE,
    approx = approx
  )

  # Reorganize output
  k <- lengths(nn$id)

  data.table::data.table(
    query_i = rep(seq_along(k), k),
    reference_j = unlist(nn$id, use.names = FALSE),
    dist = unlist(nn$dist, use.names = FALSE)
  )[dist <= radius]
}

# Check behaviour
```

```
# It is incredibly fast but fills memory: it returns 1-10M rows per second.

n = 50000 # 100-300 k points for hard cases, big ForestGEO plots
radius = 20 # something realistic, rarely > 100
p_focal = 0.05 # proportion of the forest in the focal class

xy <- cbind(x = runif(n, 0, 500), y = runif(n, 0, 500))
focal_xy <- xy[1:round(p_focal*n),]
reference_xy <- xy

system.time(d <- fast_distances_up_to_r(focal_xy, reference_xy, radius = radius, approx = 0))
```

```
      user  system elapsed
0.199    0.047    0.241
```

```
dim(d)
```

```
[1] 609387      3
```

```
head(d)
```

	query_i	reference_j	dist
	<int>	<int>	<num>
1:	1	19976	1.605451
2:	1	24611	3.469818
3:	1	1	0.000000
4:	1	5311	4.697260
5:	1	39390	5.364133
6:	1	10080	6.323878

Before other functions, let's talk about boundaries. Edge effects are important: the available area for neighborhoods is smaller near the boundary than far from it. Here we try to make the code as general as possible, and a major improvement would be to allow boundaries of arbitrary shape. The most common case, however, would be the rectangular boundary. In the updated functions, the user can define boundaries as:

- sfc polygon or multipolygon object, for boundaries of any shape. Typical parameter name: "boundary"
- as rectangular windows, expressed as a vector of (xmin, ymin, xmax, ymax) plot limits. Typical parameter name: "window"

If the user is not explicit, the functions will guess the boundary by creating a complex boundary around the set of $\{x, y\}$ points. For that, they will use the following auxiliary function. It can be handy if one has trees classified into habitats and want to restrict the analyses to within particular habitats, for example. It is a proper default but it will take time to compute, so in general it will always be better to know and define the boundary beforehand, before entering long costly loops, etc.

```
# Auxiliary function to estimate a (complex) boundary around a given set of points:
boundary_around_xy <- function(xy, q = 5) {
  xy <- as.matrix(xy)
  stopifnot(ncol(xy) == 2L, nrow(xy) >= 2L)
  storage.mode(xy) <- "double"

  # Define the buffer as a factor "q" of median distance to nearest neighbor.
  # Higher: more connected boundaries, few holes
  # Lower: more disconnected clusters and polygons
  # Very low: boundaries around each point, basically: costly and useless
  knn <- FNN::get.knn(xy, k = 1)
  buffer = q * median(knn$nn.dist[, 1], na.rm = TRUE)

  # Before entering costly polygon operations,
  # thin the input cloud of points:
  delta = buffer / 10 # used to thin the points and simplify the boundary later
  cell_x <- floor(xy[, 1] / delta)
  cell_y <- floor(xy[, 2] / delta)
  cell_id <- paste(cell_x, cell_y, sep = "_")
  xy <- xy[!duplicated(cell_id), , drop = FALSE]

  # Do the polygon operations
  pts <- sf::st_as_sf(
    as.data.frame(xy),
    coords = 1:2)

  boundary <- sf::st_buffer(pts, dist = buffer + delta)
  boundary <- sf::st_union(boundary)
  boundary <- sf::st_simplify(boundary, dTolerance = delta, preserveTopology = TRUE)
  boundary <- sf::st_make_valid(boundary)
  boundary <- sf::st_collection_extract(boundary, "POLYGON")
  boundary
}

# Check:
```

```
system.time(b <- boundary_around_xy(xy)) # this will take some time for high N
```

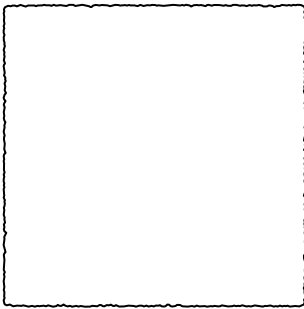
Warning in `st_collection_extract.sfc(boundary, "POLYGON")`: x is already of type POLYGON.

```
      user system elapsed  
4.143    0.206    4.458
```

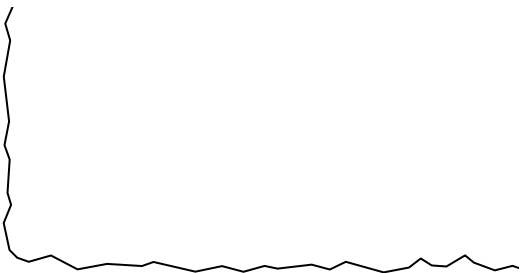
```
str(b)
```

```
sfc_POLYGON of length 1; first list element: List of 1  
 $ : num [1:425, 1:2] -0.229 -2.774 -4.474 -5.751 -4.127 ...  
- attr(*, "class")= chr [1:3] "XY" "POLYGON" "sfg"
```

```
plot(b, col = NA, border = "black", xlim = c(0, 500), ylim = c(0, 500))
```



```
plot(b, col = NA, border = "black", xlim = c(-15, 50), ylim = c(-15, 50))
```



Now, another auxiliary function to split a given set of points into “out”, “edge” and “core”, given a boundary and a distance threshold. This has a couple of useful applications in this workflow. It takes advantage of the fast calculation of distances (function above).

```

# Clasify points based on a given boundary.
# The boundary can be provided as a sfc_POLYGON of arbitrary shape
# or a rectangular window (xmin, ymin, xmax, ymax). If it is not
# provided it will be assumed rectangular around the points provided.

core_vs_edge <- function(xy,
                        boundary = NULL,
                        window = NULL,
                        radius,
                        assume_inside = FALSE,
                        approx = 0) {

  # Parameters:
  # xy: a matrix or data.frame with x and y coordinates in 1st and 2nd column respectively
  # boundary: boundary defined as a sfc_POLYGON object, for boundaries of any shape
  # window: alternative to "boundary" for rectangular boundaries, as c(xmin, ymin, xmax, ymax)
  # radius: the distance to the boundary that separates core from edge points
  # assume_inside: will accelerate if we know all xy points are inside the boundary
  # approx: precision parameter to be passed to dbscan::frNN

  # Confirm 2-columns matrix for the point coordinates
  xy <- as.matrix(xy)
  stopifnot(ncol(xy) == 2L)
  storage.mode(xy) <- "double"

  # Confirm or guess boundary
  if(is.null(boundary) & !is.null(window)) # from window to proper sf object
  {
    xmin = window[1]
    ymin = window[2]
    xmax = window[3]
    ymax = window[4]

    boundary <- sf::st_sfc(
      sf::st_polygon(list(rbind(
        c(xmin, ymin),
        c(xmax, ymin),
        c(xmax, ymax),
        c(xmin, ymax),
        c(xmin, ymin)
      )))
    )
  }
}

```

```

}

if(is.null(boundary) & is.null(window)) # guess the boundary
{
  boundary <- boundary_around_xy(xy) # inherits default "q" parameter
}

boundary <- sf::st_geometry(boundary)
boundary <- sf::st_make_valid(boundary)
boundary <- sf::st_union(boundary)

# Optional but safer unless you already know all points are inside.
if (assume_inside) {
  inside <- rep(TRUE, nrow(xy))
} else {
  pts <- sf::st_as_sf(
    as.data.frame(xy),
    coords = 1:2,
    crs = sf::st_crs(boundary),
    remove = FALSE
  )
  inside <- lengths(sf::st_intersects(pts, boundary)) > 0
}

# Densify boundary once, then use fast_distances_up_to_r()
boundary <- sf::st_boundary(boundary)
boundary <- sf::st_segmentize(boundary, dfMaxLength = radius / 10)

boundary_xy <- sf::st_coordinates(boundary)[, c("X", "Y"), drop = FALSE]
boundary_xy <- unique(boundary_xy)
storage.mode(boundary_xy) <- "double"

# Identify points near the boundary:
near <- rep(FALSE, nrow(xy))
ii <- which(inside)
if (length(ii)) {
  d <- fast_distances_up_to_r(
    query = xy[ii, , drop = FALSE],
    reference = boundary_xy,
    radius = 1.05 * radius, # small safety margin
    approx = approx
  )
}

```

```

    near[ii[unique(d$query_i)]] <- TRUE
  }

  # Build output into three classes:
  location <- rep(NA, length(inside))
  location[!inside] <- "out"
  location[inside & near] <- "edge"
  location[inside & !near] <- "core"
  location
}

# Example with a complex boundary:
library(sf)

```

Linking to GEOS 3.12.1, GDAL 3.8.4, PROJ 9.4.0; sf_use_s2() is TRUE

```

boundary <- sf::st_sfc(
  sf::st_polygon(list(rbind(
    c(100, 100),
    c(100, 400),
    c(250, 400),
    c(250, 250),
    c(400, 250),
    c(400, 100),
    c(100, 100)
  )))
)

xy <- data.frame(xy)

loc <- core_vs_edge(
  xy = xy,
  boundary = boundary,
  radius = 20
)

table(loc)

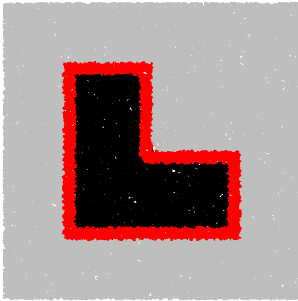
```

loc

```
core edge out
8799 4722 36479
```

```
COL <- rep("gray", ncol(xy))
COL[loc == "out"] <- "gray"
COL[loc == "edge"] <- "red"
COL[loc == "core"] <- "black"

plot(boundary, col = NA, border = "black", xlim = c(0, 500), ylim = c(0, 500))
points(xy$x, xy$y, cex = 0.1, col = COL)
```



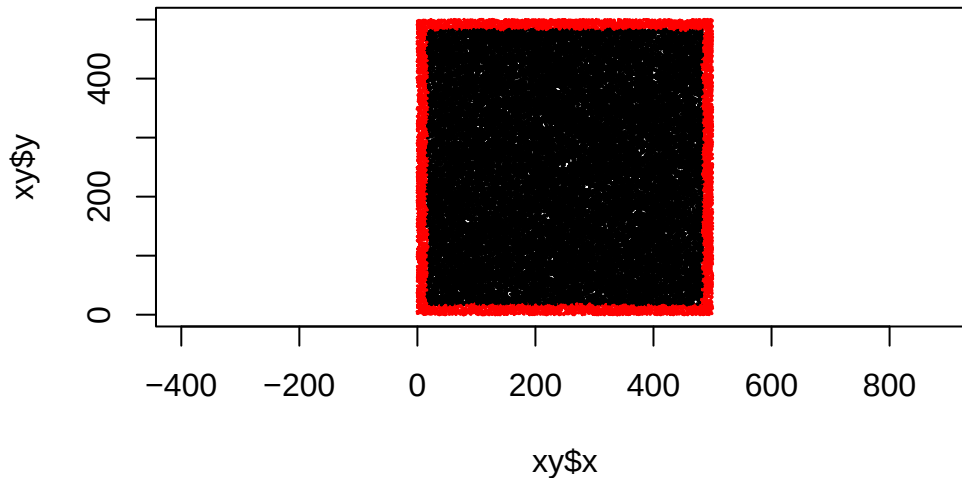
```
# It can also work by default, surrounding the provided points to guess the boundary:
loc <- core_vs_edge(xy = xy, radius = 20)
```

Warning in `st_collection_extract.sfc(boundary, "POLYGON")`: x is already of type POLYGON.

```
table(loc) # probably none out, because of the buffers
```

```
loc
core edge
43433 6567
```

```
COL <- rep("gray", ncol(xy))
COL[loc == "out"] <- "gray"
COL[loc == "edge"] <- "red"
COL[loc == "core"] <- "black"
plot(xy$x, xy$y, cex = 0.1, col = COL, asp = 1)
```



Now we look for smart ways of creating dense (but not excessively dense) grids that can be good descriptions of the shapes at the edges: intersections of circles and (possibly complex) boundaries. It is better to separate this, because in practice the user will be running loops inside loops, and we want to keep these calculations outside the costly loops.

```
# Function to make a regular grid, of a given step (~resolution), within
# a given boundary. If the boundary is not provided, it will be guessed
# from a reference "xy" dataset.
make_grid_inside_boundary <- function(boundary = NULL, window = NULL, xy = NULL, step = 999)
{
  # Parameters:
  # boundary: a sfc polygon or multipolygon object, for boundaries of any shape.
  # window: alternative to "boundary" for rectangular boundaries, as c(xmin, ymin, xmax, ymax)
  # xy: only if boundaries are not provided, they will be estimated from a reference dataset
  #      Must be a 2-columns table with x and y in 1st and 2nd column respectively.
  # step: the parameter that controls the resolution of the grid.

  # Confirm or guess boundary
  if(is.null(boundary) & !is.null(window)) # from window to proper sf object
  {
    xmin = window[1]
    ymin = window[2]
    xmax = window[3]
    ymax = window[4]

    boundary <- sf::st_sfc(
      sf::st_polygon(list(rbind(
        c(xmin, ymin),
        c(xmax, ymin),
```

```

        c(xmax, ymax),
        c(xmin, ymax),
        c(xmin, ymin)
    )))
  )
}

if(is.null(boundary) & is.null(window)) # guess the boundary
{
  xy <- as.matrix(xy)
  stopifnot(ncol(xy) == 2L)
  storage.mode(xy) <- "double"
  boundary <- boundary_around_xy(xy) # inherits default "q" parameter
}

boundary <- sf::st_geometry(boundary)
boundary <- sf::st_make_valid(boundary)
boundary <- sf::st_union(boundary)

# The grid cannot go beyond the boundaries, so the
# bounding box of the boundary is the starting point.
bb <- sf::st_bbox(boundary)

grid <- expand.grid(x = seq(from = bb["xmin"] - step, to = bb["xmax"] + step, by = step),
                  y = seq(from = bb["ymin"] - step, to = bb["ymax"] + step, by = step))

boundary <- sf::st_geometry(boundary)
boundary <- sf::st_make_valid(boundary)
boundary <- sf::st_union(boundary)
pts <- sf::st_as_sf(grid, coords = c("x", "y"), remove = FALSE)
inside <- lengths(sf::st_intersects(pts, boundary)) > 0
grid <- grid[inside,, drop = FALSE]
grid
}

# The valid resolution of the grid is not solvable with a given formula,
# as it is a form of circle + grid intersection, difficult geometrical problem.
# But it can be approached numerically, in a relatively easy way, using
# the empirical data. The idea is: calculate areas of circles for "core"
# points (that is, not affected by edge effects), and compare that with
# the known area of circles of a given radius. Once that is known,

```

```

# use a heuristic for a smaller step: that should be a conservative estimate
# that will constrain error in the corners in pseudo-worst-case scenarios.
# The user defines a tolerance: the step size derives from that,
# the radius of the circles, and the distribution of the empirical points.

guess_sufficient_grid_resolution <- function(query, boundary, radius, tol = 0.01)
{
  # query: The set of focal or query points, around which we will want to estimate
  #         available neighborhood areas.
  #         2-columns table with x and y coordinates in 1st and 2nd column, respectively.
  # radius: defines the scale of the circular neighborhoods.
  # boundary: boundary defined as a sfc_POLYGON object, for boundaries of any shape
  # window: alternative to "boundary" for rectangular boundaries, as c(xmin, ymin, xmax, ymax)
  # tol: the accepted error in area, 1% by default.

  # Confirm or guess boundary
  if(is.null(boundary) & !is.null(window)) # from window to proper sf object
  {
    xmin = window[1]
    ymin = window[2]
    xmax = window[3]
    ymax = window[4]

    boundary <- sf::st_sfc(
      sf::st_polygon(list(rbind(
        c(xmin, ymin),
        c(xmax, ymin),
        c(xmax, ymax),
        c(xmin, ymax),
        c(xmin, ymin)
      )))
    )
  }

  if(is.null(boundary) & is.null(window)) # guess the boundary
  {
    query <- as.matrix(query)
    stopifnot(ncol(query) == 2L)
    storage.mode(query) <- "double"
    boundary <- boundary_around_xy(query) # inherits default "q" parameter
  }
}

```

```

boundary <- sf::st_geometry(boundary)
boundary <- sf::st_make_valid(boundary)
boundary <- sf::st_union(boundary)

# To measure error empirically, test with the subset of
# the query that is not affected by edge effects:

# Use a coarse step to initialize
bb <- sf::st_bbox(boundary)
step = min(c((bb["ymax"] - bb["ymin"]) / 3,
             (bb["xmax"] - bb["xmin"]) / 3))

# Iterate until we reach the desired tolerance:
error = 999
while(error > tol)
{
  step = step / 2 # smaller step
  grid <- make_grid_inside_boundary(boundary = boundary, step = step)

  # Do the estimate using the core points only:
  core <- which(core_vs_edge(xy = query, boundary = boundary, radius = radius) == "core")
  d.test <- fast_distances_up_to_r(query = query[core,],
                                   reference = grid,
                                   radius = radius)
  A.estimated <- tabulate(d.test$query_i, nbins = length(core)) * step^2
  A.true = pi*radius^2
  error = mean(abs(as.numeric(A.estimated) - A.true) / A.true)
  #print(error)
}

# As a general rule, error will be larger near the boundary,
# and greater if the boundary is very complicated.
# So, to apply this, we need greater resolution (smaller steps).
# How smaller? Well... there is not a neat answer for that.
# But ChatGPT says that this is a useful conservative heuristic,
# derived from Steiner / parallel-body argument:
K = 1 + 4 / pi
suggested_step = step / K
ppc = floor(pi*(radius^2)/(suggested_step^2)) # points per circle

# There are other rules of thumb and analytical results for other cases:

```

```

conservative_step = tol * radius / (2 * sqrt(2)) # to constrain maximum error
less_conservative_step = radius * (2 * tol)^(2/3) # to constrain average error

# The output with everything of value:
c(suggested_step = suggested_step,
  conservative_step = conservative_step,
  less_conservative_step = less_conservative_step,
  radius = radius, tol = tol, error = error, points_per_circle = ppc)
}

```

To calculate the corrected area of the neighborhoods, while saving computation time, we have to combine (1) the standard formula for the circle area for the core focal points, and (2) the grid-derived estimates near the edge. For this second point, we cross: (a) the subset of focal points closer than R to the boundary, and (b) the subset of grid points closer than $2R$ to the boundary, being R the radius of the scale being evaluated.

```

get_area_of_neighborhoods <- function(query = NULL,
                                       boundary = NULL,
                                       window = NULL,
                                       radius = NULL,
                                       step = NA,
                                       tol = 0.01,
                                       grid_for_edge = NULL)

{
  # *** describe parameters
  # *** solve window and boundary properly

  # Make the grid.
  # The step parameter defines the grid resolution.
  # If not provided, will be derived from the tolerance.
  if(is.na(step))
  {
    errors.etc <- guess_sufficient_grid_resolution(xy = query,
                                                  boundary = boundary,
                                                  radius = radius,
                                                  tol = tol)

    step = errors.etc["suggested_step"]
  }

  grid <- make_grid_inside_boundary(boundary, step = step)
}

```

```

# Split the both the query and the grid as core vs. edge
ceo_query <- core_vs_edge(xy = query, boundary = boundary, radius = radius)
query_xy_edge <- query_xy[ceo_query == "edge",,drop = FALSE]

if(is.null(grid_for_edge))
{
  # In most practical applications, we probably want this to be calculated
  # outside, so we avoid repetitions during loops and so on.
  ceo_grid <- core_vs_edge(xy = grid, boundary = boundary, radius = 2*radius)
  grid_for_edge <- grid[ceo_grid == "edge",,drop = FALSE]
}

# Now take advantage of the neighbor-identifying workhorse:
d <- fast_distances_up_to_r(query = query_xy_edge,
                           reference = grid_for_edge,
                           radius = radius)

# Put all the areas together: initialize with full circles,
# then overwrite with the areas calculated for the edge cases:
A <- rep(pi*radius^2, nrow(query))
A[ceo_query == "edge"] <- tabulate(d$query_i, nbins = nrow(query_xy_edge)) * step^2
A
}

#plot(grid_for_edge, cex = 0.1, asp = 1)

#
#
#
# *** generalize and make examples with different abundance metrics

if(FALSE)
{
  # Count the number of neighbors and express that as trees / area:
  d <- fast_distances_up_to_r(focal_xy, reference_xy, radius, approx = 0)
  number_of_neighbors <- tabulate(d$query_i, nbins = nrow(focal_xy))
  barplot(table(number_of_neighbors)) # if random locations: Poisson ~ normal distribution

  # Express as densities, edges corrected:
  length(number_of_neighbors) == length(A)
  neighbors_per_area <- number_of_neighbors / A
}

```

```
hist(neighbors_per_area)
}
```

Worked example

The ring case

Neighborhoods can be defined as circles (everything from the focal point to a given distance) or rings (things further away than “r” but closer than “R” to the focal point). Rings are conceptually more explicit about the scale at which those measurements apply. If you are doing this type of analysis you should know which option (circular vs ring-shaped neighborhoods) you prefer.

Comparing observations with expectations

I present examples on how to compare a given point pattern with the expectations.

First, I present the Ripley’s K and Besag’s L case, using standard R implementation. These are based on counts and Poisson point pattern as the null expectation (that is, independent and uniformly random distribution of points). This functionality was in the original CTFS package but is not implemented directly here as part of the functions in fgeo2.

Second, I present a template of a more general approach based on numerical null models

Example of Ripley’s K

fgeo2 does not contain Ripley’s K and there is no need for it, as this is a standard spatial analytical tool that is well-maintained in other packages. Instead, I include an example on how you would implement this analysis using {x, y} coordinates.

```
# Example for the living trees of one species
# At what scales is Faramaea occidentalis in BCI spatially aggregated?

sp = "faraoc"
```

Example of numerical null model

Numerical null models are very flexible, and could be used for the same purposes as Ripley's K, with added advantages:

- Ripley's K uses the simplest null model (uniform random distribution = Poisson point pattern), but in a customizable numerical null model the user could incorporate different null models, like the torus translation or constrained torus translation.
- The metric of interest in a null model approach could be any, not just counts of neighbors as implicit in Ripley's K. It could be biomass, sum of basal area, etc.
- Null models already incorporate the expectation of fewer neighbors for focal points near the edges, so they can be a direct way of correcting for edge effects for boundaries or arbitrary shape.

In general, null models have nice properties for generalization and flexibility. Their main practical limitations are: (1) the tremendous responsibility on the definition of a reasonable / useful expectation, and (2) computation and storage. If you are afraid of responsibility, do not use null models.

```
# Null models compare observations (one instance) with expectations (many instances).  
# If reality does not deviate much from the expectations by the null model, then  
# the researcher cannot reject the null hypothesis modelled by the null model.  
# If reality deviates from the expectations, then we can reject the null hypothesis.
```